

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP MÔI TRƯỜNG

TRƯỜNG ĐẠI HỌC THỦY LỢI



BÁO CÁO MÔN HỌC

Đề tài : Dự đoán mức độ nghiêm trọng tai nạn giao thông theo thời tiết

Giảng viên hướng dẫn: TS. Tạ Quang Chiêu

Nhóm sinh viên :

Hồ Đức Minh - 2351260669

Lê Văn Chương - 2351260643

Lưu Minh Quân - 2351260684

Hà Nội, 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN.....	2
1.1 Giới thiệu đề tài	3
1.2 Mục tiêu nghiên cứu	3
1.3 Yêu cầu kĩ thuật và phạm vi đề tài.....	4
1.4 Đối tượng và phương pháp	4
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ.....	4
2.1 Tổng quan về tiền xử lý dữ liệu	5
2.2 Các kỹ thuật tiền xử lý dữ liệu cốt lõi.....	5
2.2.1. Xử lý giá trị khuyết thiếu	5
2.2.2. Loại bỏ giá trị ngoại lai bằng phương pháp IQR	5
2.3. Chuyển đổi và chuẩn hóa đặc trưng.....	6
2.3.1. Mã hóa biến phân loại bằng StringIndexer.....	6
2.3.2. Chuẩn hóa dữ liệu bằng StandardScaler	6
2.4. Xử lý mất cân bằng dữ liệu bằng trọng số lớp.....	6
2.5. Lựa chọn đặc trưng bằng kiểm định Chi-Square	7
2.6. Thuật toán phân loại Random Forest.....	7
CHƯƠNG 3: TRIỂN KHAI	7
3.1 Tổng quan dữ liệu	8
3.1.1. Nguồn dữ liệu và quy mô	8
3.1.2. Các trường dữ liệu chính và ý nghĩa đặc trưng.....	8
3.2 Tích hợp và tổng hợp dữ liệu	9
3.2.1 Quy trình nạp và tinh lọc dữ liệu thành phần	9
3.2.2. Kỹ thuật tích hợp dữ liệu và xử lý thời gian	10

3.2.3. Hợp nhất dữ liệu lịch sử và dữ liệu tạm thời	10
3.2.4. Lưu trữ dữ liệu tối ưu hóa định dạng Parquet.....	11
3.3. Phân tích dữ liệu	12
3.4. Quy trình tiền xử lý.....	16
3.4.1. Xử lý giá trị khuyết thiếu, giá trị trùng lặp và chuẩn hóa dữ liệu lỗi.....	16
3.4.2. Rời rạc hóa biến thời gian.....	19
3.4.4. Mã hóa đặc trưng phân loại bằng StringIndexer.....	19
3.4.5. Xử lý Outlier	20
3.4.6. Xử lý dữ liệu mất cân bằng.....	21
3.4.7. Chuẩn hóa dữ liệu	22
3.4.8. Chia tập train/ test bằng Stratified Sampling.....	23
3.4.9. Kiểm định Chi-bình phương.....	24
3.5. Mô hình học máy Random Forest	25
3.5.1. Chọn mô hình và dữ liệu đầu vào.	25
3.5.2. Quy trình huấn luyện và cấu hình tham số.	25
3.5.3. Đánh giá hiệu suất mô hình.	26
CHƯƠNG 4: ĐÁNH GIÁ HỆ THỐNG VÀ THẢO LUẬN KẾT QUẢ.....	27
4.1. Đánh giá kiến trúc tổng thể của hệ thống	27
4.2. Khả năng dự báo và ý nghĩa thực tiễn	28
4.3. Hạn chế và hướng phát triển tương lai.....	28

CHƯƠNG 1: GIỚI THIỆU BÀI TOÁN

1.1 Giới thiệu đề tài

Trong thời đại số ngày nay, sự phát triển mạnh mẽ của dữ liệu giao thông đã cung cấp thông tin quý giá để nâng cao an toàn giao thông. Các mô hình học máy được sử dụng để dự đoán mức độ nghiêm trọng của tai nạn giao thông. Tai nạn giao thông không chỉ là vấn đề kỹ thuật mà còn là một thách thức xã hội cấp thiết, gây ra thiệt hại lớn về người và tài sản trên toàn cầu.

Một yếu tố quan trọng nhưng thường rất phức tạp và biến động ảnh hưởng đến an toàn giao thông là điều kiện thời tiết và môi trường xung quanh. Hiểu rõ mối quan hệ giữa thời tiết và mức độ nghiêm trọng của tai nạn là điều cần thiết để các cơ quan quản lý đưa ra cảnh báo kịp thời.

Từ thực tế đó, đề tài “Dự đoán mức độ nghiêm trọng tai nạn giao thông theo thời tiết” đã được thực hiện. Mục tiêu là khai thác bộ dữ liệu tai nạn trong quá khứ để xây dựng một mô hình học máy có khả năng dự báo mức độ thương vong dựa trên các yếu tố ngoại cảnh như thời tiết.

1.2 Mục tiêu nghiên cứu

Mục tiêu của nghiên cứu này là xây dựng một quy trình tiền xử lý dữ liệu hoàn chỉnh và mô hình phân loại hiệu quả trên nền tảng dữ liệu lớn. Trước hết, sử dụng các bước tiền xử lý tập trung vào việc làm sạch và chuẩn hóa bộ dữ liệu tai nạn giao thông giai đoạn 1979-2025, xử lý các vấn đề của dữ liệu thực tế như giá trị khuyết thiếu, dữ liệu nhiễu và sự mất cân bằng giữa các lớp dữ liệu [1]. Tiếp theo, sử dụng các kỹ thuật phân tích sẽ xác định những yếu tố thời tiết và điều kiện mặt đường có ảnh hưởng mạnh nhất đến mức độ nghiêm trọng của tai nạn. Cuối cùng, nghiên cứu hướng tới việc áp dụng thuật toán Random Forest để dự đoán chính xác mức độ thương vong, cung cấp công cụ hỗ trợ ra quyết định cho các hệ thống giao thông thông minh trong tương lai.

1.3 Yêu cầu kĩ thuật và phạm vi đề tài

Dữ liệu sử dụng là bộ dữ liệu tổng hợp về va chạm và thương vong giao thông tại Vương quốc Anh, được lưu trữ dưới định dạng parquet để tối ưu hóa hiệu năng truy xuất trong môi trường tính toán phân tán. Dữ liệu bao gồm nhiều khía cạnh bao gồm điều kiện ánh sáng, loại đường, giới hạn tốc độ và các biến số về thời tiết như mưa, gió mạnh hay tuyết rơi. Về mặt kỹ thuật, hệ thống đòi hỏi khả năng xử lý dữ liệu quy mô lớn sử dụng thư viện PySpark, thực hiện các tác vụ từ trích xuất đặc trưng (Feature Engineering) đến mã hóa biến phân loại bằng StringIndexer và chuẩn hóa dữ liệu số bằng StandardScaler. Quy trình đánh giá mô hình sẽ dựa trên các chỉ số đo lường học máy tiêu chuẩn.

1.4 Đối tượng và phương pháp

Đối tượng nghiên cứu là các bản ghi dữ liệu va chạm giao thông chứa thông tin về điều kiện môi trường và mức độ thương vong của nạn nhân. Phương pháp được thực hiện theo các kỹ thuật tiền xử lý dữ liệu và thuật toán học máy có giám sát. Phương pháp sử dụng mô hình lập trình của SparkSession để thực hiện các bước xử lý song song, giúp tối ưu hóa thời gian tính toán trên tập dữ liệu. Bên cạnh đó, các phương pháp thống kê như kiểm định Chi-Square cũng được áp dụng để đánh giá mối tương quan giữa các đặc trưng đầu vào, từ đó tinh lọc bộ dữ liệu nhằm đạt được hiệu suất dự báo cao nhất cho mô hình Random Forest.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Tổng quan về tiền xử lý dữ liệu

Tiền xử lý dữ liệu là một bước quan trọng trong quy trình khai phá tri thức, đặc biệt đối với những bộ dữ liệu thô chứa nhiều nhiễu như dữ liệu tai nạn giao thông. Các bước tiền xử lý bao gồm các giai đoạn từ làm sạch, tích hợp, chuyển đổi cho đến giảm chiều dữ liệu nhằm tạo ra một bản dữ liệu sạch cho mô hình học máy. Việc áp dụng các kỹ thuật tiền xử lý không chỉ giúp cải thiện hiệu suất tính toán mà còn đảm bảo tính giải thích được của các kết quả dự báo [2].

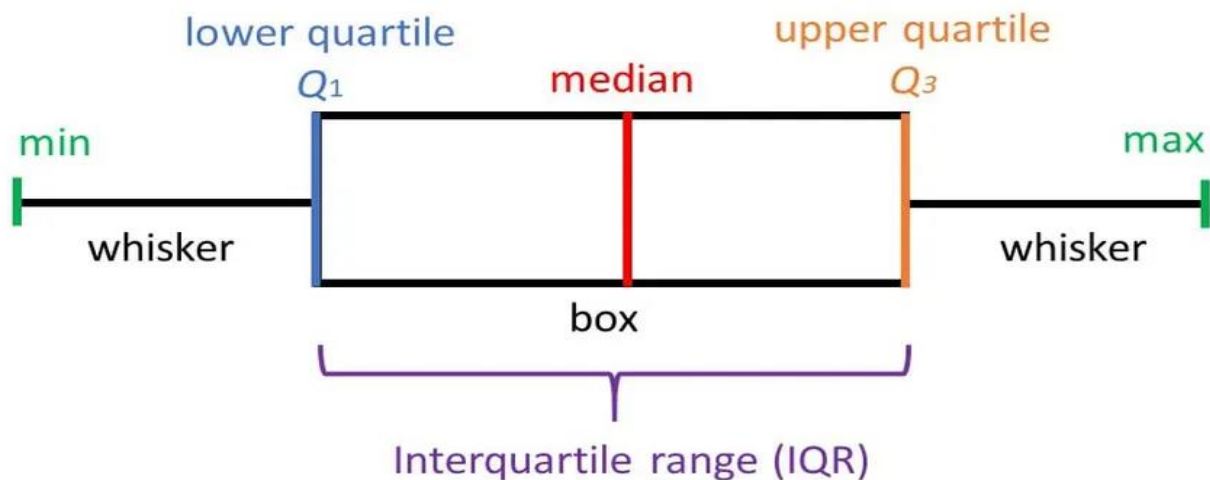
2.2 Các kỹ thuật tiền xử lý dữ liệu cốt lõi

2.2.1. Xử lý giá trị khuyết thiếu

Trong thực tế, các biến số như tuổi của nạn nhân hay giới hạn tốc độ thường xuất hiện các giá trị trống hoặc giá trị lỗi (được ký hiệu là -1 trong bộ dữ liệu). Đối với các biến định lượng, sử dụng kỹ thuật gán giá trị bằng trung vị (Median) được ưu tiên sử dụng để tránh sự chênh lệch do các giá trị nhiễu gây ra. Đối với các biến định danh như điều kiện thời tiết, sử dụng phương pháp yếu vị (Mode) được áp dụng để giữ nguyên đặc tính phân phối của dữ liệu gốc [3]. Công thức xác định trung vị của dãy số đã sắp xếp có n phần tử được xác định là giá trị tại vị trí $\lfloor \frac{n}{2} \rfloor$ nếu n lẻ, hoặc trung bình cộng của hai giá trị giữa nếu n chẵn.

2.2.2. Loại bỏ giá trị ngoại lai bằng phương pháp IQR

Giá trị ngoại lai (Outliers) có thể làm sai lệch các thông số thống kê của mô hình. Sử dụng phương pháp Khoảng biến thiên tứ phân vị (Interquartile Range - IQR) để xác định các điểm dữ liệu bất thường. Khoảng IQR được tính bằng hiệu số giữa tứ phân vị thứ ba (Q_3) và tứ phân vị thứ nhất (Q_1): $IQR = Q_3 - Q_1$. Dựa trên đó, các giá trị nằm ngoài phạm vi $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$ sẽ bị loại bỏ nhằm đảm bảo tính tập trung của dữ liệu [2].



Hình 2.1: Mô hình xác định giá trị ngoại lai dựa trên khoảng biến thiên tứ phân vị IQR

2.3. Chuyển đổi và chuẩn hóa đặc trưng

2.3.1. Mã hóa biến phân loại bằng *StringIndexer*

Các thuật toán học máy như Random Forest yêu cầu đầu vào là dạng số, nên việc sử dụng *StringIndexer* trong PySpark để chuyển đổi các nhãn văn bản thành các chỉ số. Việc này gán cho mỗi giá trị duy nhất trong cột một số nguyên dựa trên tần suất xuất hiện, giúp mô hình học máy hiểu được cấu trúc của các biến định danh như loại đường hay giới hạn tốc độ mà không làm mất đi thông tin phân loại [4].

2.3.2. Chuẩn hóa dữ liệu bằng *StandardScaler*

Để tránh sự khác biệt về đơn vị đo lường giữa các biến (ví dụ: tuổi từ 0-100 và tốc độ từ 20-70), kỹ thuật chuẩn hóa Z-score (Standardization) được sử dụng. Công thức biến đổi một giá trị x thành giá trị chuẩn hóa z như sau: $z = \frac{x - \mu}{\sigma}$, trong đó μ là giá trị trung bình và σ là độ lệch chuẩn của đặc trưng đó. Kết quả của quá trình này đưa dữ liệu về phân phối có trung bình bằng 0 và độ lệch chuẩn bằng 1, giúp thuật toán hội tụ nhanh hơn và công bằng hơn giữa các đặc trưng [2].

2.4. Xử lý mất cân bằng dữ liệu bằng trọng số lớp

Một vấn đề lớn trong dữ liệu tai nạn là số lượng các vụ tai nạn nghiêm trọng thường ít hơn rất nhiều so với các vụ va chạm nhẹ. Để giải quyết vấn đề này mà không làm mất thông tin như phương pháp lấy mẫu, nghiên cứu sử dụng kỹ thuật tính trọng số lớp (Class Weights). Trọng số W_i cho lớp i được tính theo công thức: $W_i = \frac{N_{max}}{N_i}$, trong đó N_{max} là số lượng mẫu của lớp chiếm ưu thế nhất và N_i là số mẫu của lớp thứ i . Việc join trọng số này vào dữ liệu huấn luyện giúp mô hình chú trọng hơn đến các lớp thiểu số nhưng quan trọng [3].

2.5. Lựa chọn đặc trưng bằng kiểm định Chi-Square

Kiểm định Chi-Square (χ^2) được sử dụng để đánh giá mối quan hệ độc lập giữa các biến phân loại và mức độ nghiêm trọng của tai nạn. Công thức tính giá trị kiểm định là: $\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i}$, với O_i là tần số quan sát được và E_i là tần số mong đợi. Dựa trên giá trị p-value thu được, nghiên cứu có thể loại bỏ các đặc trưng không có ý nghĩa thống kê, từ đó làm giảm chiều dữ liệu và tránh hiện tượng quá khớp (overfitting) [2].

2.6. Thuật toán phân loại Random Forest

Random Forest là một phương pháp học máy mạnh mẽ dựa trên việc kết hợp nhiều cây quyết định độc lập. Thuật toán sử dụng kỹ thuật đóng bao (Bagging) để lấy mẫu dữ liệu và lựa chọn ngẫu nhiên một tập con các đặc trưng tại mỗi nút chia của cây. Kết quả cuối cùng là sự tổng hợp (Majority Voting) từ tất cả các cây thành phần, giúp giảm thiểu phương sai và tăng cường độ chính xác cho bài toán dự báo phức tạp [5].

Để đánh giá hiệu suất của mô hình Random Forest, và với các bộ dữ liệu có hiện tượng mất cân bằng lớp chỉ số F1-Score là lựa chọn được sử dụng.

hái niệm, F1-Score là trung bình điều hòa giữa hai độ đo Precision (Độ chính xác - tỷ lệ dự báo đúng trong tổng số các dự báo thuộc lớp đó) và Recall (Độ thu hồi / Độ bao phủ - tỷ lệ nhận diện đúng trong tổng số các mẫu thực tế thuộc lớp đó). Việc sử dụng trung bình điều hòa giúp F1-Score đánh giá khách quan hơn, đảm bảo mô hình phải cân bằng được

giữa việc không báo động nhầm (False Positive) và không bỏ sót các trường hợp thực tế (False Negative).

Công thức tính F1.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Công thức tính Precision và Recall:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

CHƯƠNG 3: TRIỂN KHAI

3.1 Tổng quan dữ liệu

3.1.1. Nguồn dữ liệu và quy mô

Hệ thống khai thác bộ dữ liệu an toàn đường bộ mở từ Chính phủ Vương quốc Anh, một nguồn tài nguyên chuẩn hóa và có độ tin cậy cao trong lĩnh vực phân tích dữ liệu giao thông. Tập dữ liệu này được tích hợp từ hai nguồn chính là tệp thống kê thương vong Casualties và tệp thống kê va chạm Collisions giai đoạn 1979-2025. Tổng quy mô dữ liệu sau khi hợp nhất đạt hàng chục triệu bản ghi, cho phép mô hình học máy tiếp cận được đa dạng các kịch bản tai nạn trong nhiều điều kiện môi trường và bối cảnh xã hội khác nhau qua gần năm thập kỷ.

3.1.2. Các trường dữ liệu chính và ý nghĩa đặc trưng

Trong quá trình thực hiện, hệ thống tập trung vào các nhóm biến số cốt lõi có tác động trực tiếp đến mục tiêu dự báo. Nhóm biến mục tiêu (Label) chính là mức độ nghiêm trọng của thương vong (casualty_severity), đóng vai trò là biến phân loại để mô hình học tập.

Nhóm thông tin về nạn nhân bao gồm các cột như loại đối tượng gặp nạn (casualty_class), giới tính (sex_of_casualty), độ tuổi (age_of_casualty) và loại hình phương tiện liên quan (casualty_type). Những thông tin này giúp hệ thống xác định mức độ tổn thương của các nhóm đối tượng khác nhau khi xảy ra va chạm đường bộ.

Nhóm đặc trưng về điều kiện môi trường và hạ tầng đóng vai trò quyết định trong đề tài dự báo theo thời tiết. Các cột dữ liệu chính bao gồm điều kiện thời tiết (weather_conditions), tình trạng ánh sáng (light_conditions), tình trạng mặt đường (road_surface_conditions), giới hạn tốc độ tại khu vực (speed_limit) và đặc điểm phân loại đường (road_type). Ngoài ra, các yếu tố về không gian và thời gian cũng được tích hợp chặt chẽ thông qua các trường như khu vực đô thị hay nông thôn (urban_or_rural_area), ngày trong tuần (day_of_week) và thời điểm xảy ra tai nạn (time). Việc kết hợp các nhóm đặc trưng này tạo nên một bức tranh toàn diện về bối cảnh xảy ra tai nạn, từ đó hỗ trợ mô hình Random Forest đưa ra những dự báo chính xác về rủi ro dựa trên điều kiện khí tượng thực tế.

3.2 Tích hợp và tổng hợp dữ liệu

3.2.1 Quy trình nạp và tinh lọc dữ liệu thành phần

Giai đoạn đầu tiên của quy trình là nạp và chuẩn hóa các tệp dữ liệu thô từ định dạng CSV vào hệ thống tính toán phân tán Spark. Dữ liệu thương vong (df_casualty) với quy mô hơn 11,9 triệu bản ghi được lọc bỏ các cột không cần thiết, chỉ giữ lại các trường thông tin cốt lõi như collision_index, casualty_class, sex_of_casualty, age_of_casualty, casualty_severity và casualty_type. Tương tự, dữ liệu về các vụ va chạm (df_collision) với hơn 9 triệu bản ghi cũng được tinh lọc để giữ lại bối cảnh về điều kiện mặt đường, ánh sáng và thời tiết. Việc tinh lọc này không chỉ giúp giảm bớt gánh nặng tính toán cho hệ thống mà còn tập trung vào các đặc trưng có giá trị cao nhất cho bài toán dự báo mức độ nghiêm trọng của tai nạn giao thông.



```

1 df_casualty_1979_2024 = df_casu_drop_col.select(
2     F.col("collision_index"),
3     F.col("casualty_class").cast(IntegerType()),
4     F.col("sex_of_casualty").cast(IntegerType()),
5     F.col("age_of_casualty").cast(IntegerType()),
6     F.col("casualty_severity").cast(IntegerType()),
7     F.col("casualty_type").cast(IntegerType()),
8 )

```

Hình 3.1: Thực hiện ép kiểu dữ liệu casualty



```

1 df_collision_1979_2024 = df_collision_drop_col.select(
2     F.col("collision_index"),
3     F.col("collision_severity").cast(IntegerType()),
4     F.to_date(F.col("date"), "dd/MM/yyyy").alias("date"),
5     F.col("day_of_week").cast(IntegerType()),
6     F.split(F.col("time"), ":")[0].cast(IntegerType()).alias("time"),
7     F.col("road_type").cast(IntegerType()),
8     F.col("speed_limit").cast(IntegerType()),
9     F.col("light_conditions").cast(IntegerType()),
10    F.col("weather_conditions").cast(IntegerType()),
11    F.col("road_surface_conditions").cast(IntegerType()),
12    F.col("urban_or_rural_area").cast(IntegerType()),
13 )

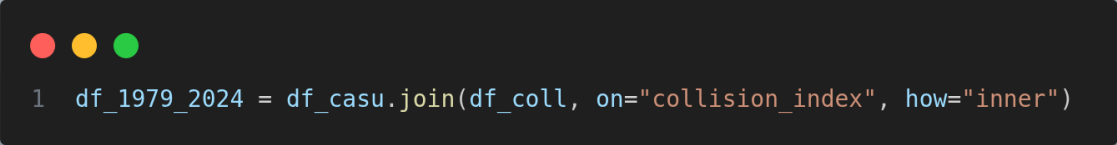
```

Hình 3.2: Thực hiện ép kiểu dữ liệu collision

3.2.2. Kỹ thuật tích hợp dữ liệu và xử lý thời gian

Để xây dựng một tập dữ liệu huấn luyện hoàn chỉnh, hệ thống thực hiện phép kết hợp nội (Inner Join) giữa bảng thương vong và bảng va chạm dựa trên thuộc tính khóa là

collision_index. Trước khi thực hiện kết hợp, các cột dữ liệu cần được ép kiểu (Casting) về dạng số nguyên hoặc ngày tháng để đảm bảo tính nhất quán. Đặc biệt, cột thời gian được xử lý bằng kỹ thuật tách chuỗi (F.split) để trích xuất giá trị giờ, trong khi cột ngày tháng được định dạng lại bằng hàm F.to_date theo chuẩn yyyy-MM-dd. Quy trình này giúp hợp nhất bối cảnh môi trường của vụ tai nạn với thông tin chi tiết của từng nạn nhân, tạo nên một bản ghi tổng hợp giàu thông tin cho mô hình học máy.

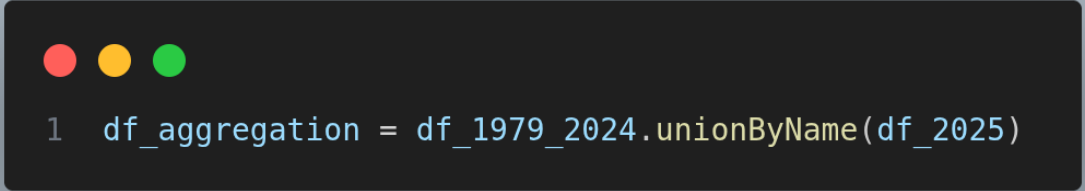


```
1 df_1979_2024 = df_casu.join(df_coll, on="collision_index", how="inner")
```

Hình 3.3: Thực thi phép join giữa 2 bảng casualty và collision trên nền tảng spark

3.2.3. Hợp nhất dữ liệu lịch sử và dữ liệu tạm thời

Việc xử lý dữ liệu từ nhiều giai đoạn khác nhau với sự khác biệt nhỏ về cấu trúc. Hệ thống đã thực hiện quy trình nạp dữ liệu lịch sử (1979-2024) và dữ liệu tạm thời của năm 2025 theo các bước chuẩn hóa tương đồng, sau đó sử dụng hàm unionByName để hợp nhất hai tập dữ liệu này. Kỹ thuật này đảm bảo rằng các cột dữ liệu được khớp nối chính xác theo tên, ngay cả khi thứ tự các cột trong tệp nguồn có sự thay đổi. Kết quả cuối cùng là một tập dữ liệu tổng hợp (Aggregation) xuyên suốt từ năm 1979 đến năm 2025, phản ánh đầy đủ xu hướng và đặc điểm của tai nạn giao thông qua nhiều thập kỷ.



```
1 df_aggregation = df_1979_2024.unionByName(df_2025)
```

Hình 3.4: Nối 2 bảng dữ liệu lại từ 1979 – 2024 với 2025

3.2.4. Lưu trữ dữ liệu tối ưu hóa định dạng Parquet

Sau khi hoàn tất quá trình tổng hợp và hợp nhất, toàn bộ dữ liệu được ghi xuống bộ nhớ dưới định dạng Parquet với chế độ overwrite để tối ưu hóa hiệu năng. Định dạng Parquet là lựa chọn lý tưởng cho các bài toán Big Data nhờ khả năng nén dữ liệu theo cột, giúp giảm đáng kể dung lượng lưu trữ trên đĩa so với định dạng CSV truyền thống. Hơn nữa, Parquet còn cho phép bảo toàn các kiểu dữ liệu phức tạp và cấu trúc lược đồ (Schema), giúp các giai đoạn tiền xử lý và huấn luyện mô hình ở các chương tiếp theo có thể truy xuất dữ liệu một cách nhanh chóng và ổn định nhất.



Hình 3.5: Lưu file dưới dạng parquet

3.3. Phân tích dữ liệu

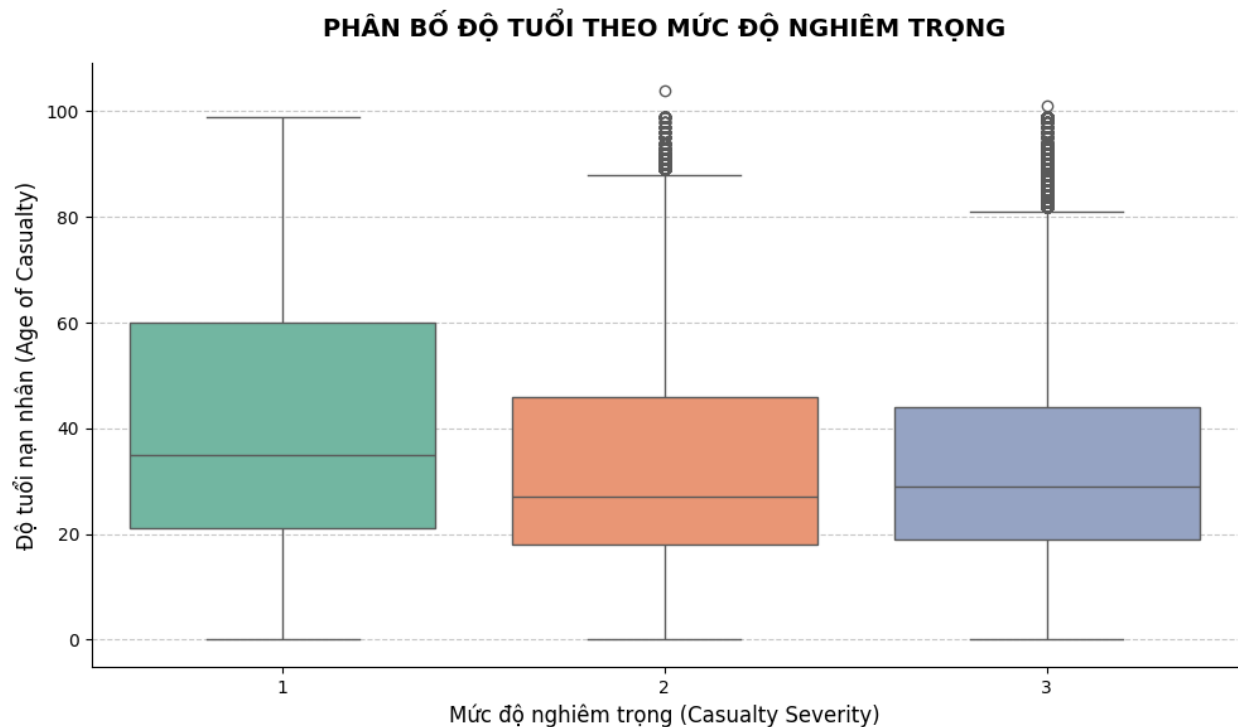
	Features	Missing values
15	urban_or_rural_area	4837106
3	age_of_casualty	218553
14	road_surface_conditions	17010
2	sex_of_casualty	11940
13	weather_conditions	3350
12	light_conditions	2621
11	speed_limit	358
10	road_type	124
7	date	10
9	time	10
1	casualty_class	1

Hình 3.6. Số lượng dữ liệu thiếu



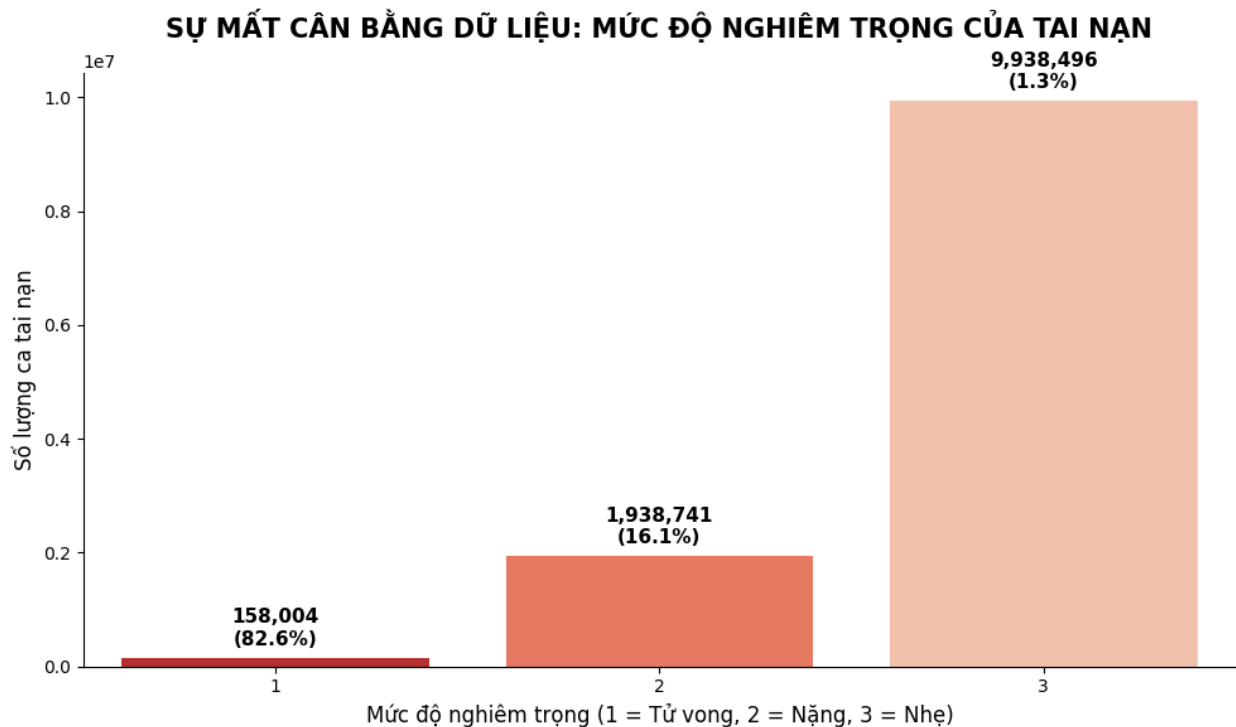
Hình 3.7. Biểu đồ cột thể hiện số lượng dữ liệu khuyết thiếu (triệu dòng)

Dựa vào biểu đồ và hình 3.6, tập dữ liệu hiện có 11 đặc trưng bị khuyết giá trị. Tình trạng này tập trung nghiêm trọng nhất ở 4 giá trị là “Ở thành thị hay nông thôn” (urban_or_rural_area) với 4.837.106 dòng và “Tuổi của nạn nhân” (age_of_casualty) với 218.553 dòng, “Điều kiện mặt đường” (road_surface_conditions) với 17.010 dòng, “Giới tính nạn nhân” (sex_of_casualty) với 11.940 dòng. 7 biến còn lại bao gồm weather_conditions, light_conditions, speed_limit, road_type, date và time có lượng thiếu hụt không đáng kể. Sự phân bố này đòi hỏi nhóm phải ưu tiên áp dụng các kỹ thuật tiền xử lý phù hợp như điền khuyết hoặc loại bỏ đối với các đặc trưng môi trường, nhằm đảm bảo chất lượng dữ liệu đầu vào cho mô hình huấn luyện.



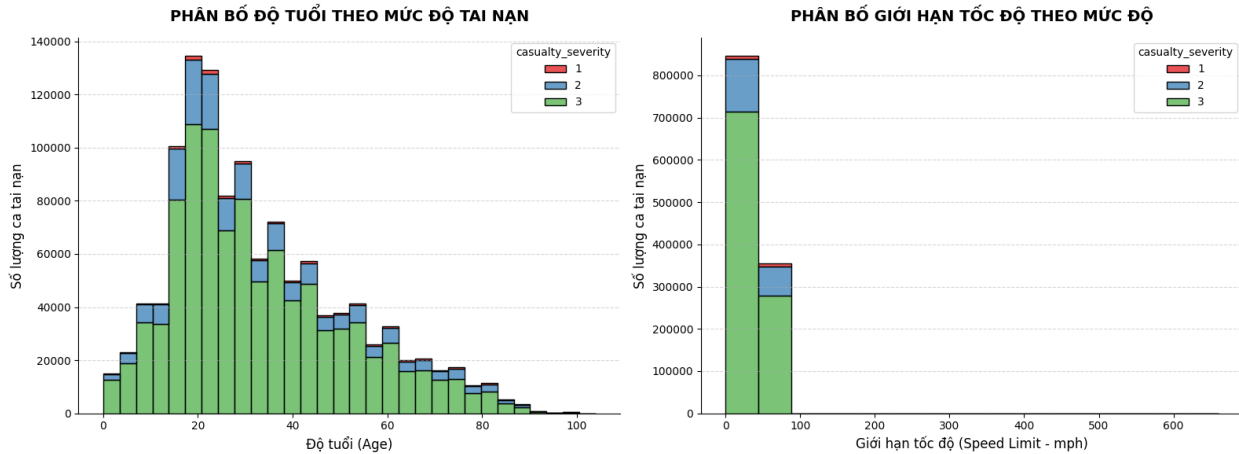
Hình 3.7. Biểu đồ Box Plot thể hiện sự phân bố độ tuổi nạn nhân dựa trên mức độ nghiêm trọng tai nạn

Qua biểu đồ Box Plot, phân bố độ tuổi nạn nhân ở cả ba mức độ tai nạn khá tương đồng. Độ tuổi trung vị đều xoay quanh mốc 30 tuổi, với phần lớn dữ liệu (khoảng IQR) tập trung từ 20 đến 50 tuổi. Điểm đáng lưu ý nhất là sự xuất hiện dày đặc của các biến ngoại lai (outliers) ở dải tuổi rất cao (từ 80 đến xấp xỉ 120 tuổi), cho thấy khả năng có lỗi nhập liệu cần được tiến hành làm sạch. Nhìn chung, do không có sự khác biệt rõ rệt về phân bố tuổi giữa các nhóm, đặc trưng này có thể không mang lại trọng số quá lớn trong việc dự đoán mức độ nghiêm trọng của tai nạn.



Hình 3.8. Biểu đồ cột thể hiện số lượng ca tai nạn theo mức độ nghiêm trọng

Dựa vào biểu đồ phân bố mức độ nghiêm trọng của tai nạn, tập dữ liệu đang gặp phải tình trạng mất cân bằng (class imbalance) cực kỳ nghiêm trọng. Cụ thể, số lượng ca tai nạn nhẹ (mức 3) chiếm ưu thế tuyệt đối với hơn 9,9 triệu dòng, tương đương 82,6% tổng dữ liệu. Trong khi đó, các ca tai nạn nặng (mức 2) và tử vong (mức 1) lại thuộc nhóm thiểu số với tỷ lệ rất nhỏ, lần lượt chỉ chiếm 16,1% và 1,3%. Sự chênh lệch quá lớn này chắc chắn sẽ khiến mô hình học máy bị thiên lệch (bias), dẫn đến xu hướng học vẹt và dự đoán mọi trường hợp đều là tai nạn nhẹ. Do đó, việc áp dụng các kỹ thuật tái lấy mẫu như SMOTE (Oversampling) hoặc Undersampling ở bước tiền xử lý tiếp theo là bắt buộc để giúp mô hình có thể nhận diện chính xác các ca tử vong hiếm gặp.



Hình 3.9: Phân bố độ tuổi và giới hạn tốc độ theo mức độ tai nạn

Thông qua biểu đồ phân bố độ tuổi nạn nhân, hình dáng đồ thị lệch phải (right-skewed), với đỉnh tập trung cao nhất ở nhóm thanh niên và trung niên, đặc biệt là dải tuổi từ 20 đến 30. Tần suất xảy ra tai nạn giảm dần khi độ tuổi tăng lên. Dù có sự khác biệt lớn về tổng số lượng ca giữa các nhóm mức độ (phần màu xanh lá chiếm đa số), tỷ lệ số ca tử vong và bị thương nặng (màu đỏ và xanh dương) trên tổng số tai nạn vẫn xuất hiện xuyên suốt các nhóm tuổi, không tập trung đột biến ở một độ tuổi cụ thể nào, cho thấy mức độ tai nạn phụ thuộc vào nhiều yếu tố khách quan khác bên cạnh độ tuổi.

Biểu đồ phân bố giới hạn tốc độ cho thấy sự tập trung dữ liệu rõ rệt ở khoảng tốc độ 0-100 mph. Trong bộ dữ liệu, mốc 30 mph ghi nhận số lượng tai nạn áp đảo (hơn 6 triệu ca), chủ yếu là tai nạn nhẹ, điều này phù hợp với đặc điểm giao thông đông đúc khu vực nội đô. Tuy nhiên, ở các mốc tốc độ cao hơn, gần với mốc 100 mph (đường quốc lộ, cao tốc), tỷ lệ các ca tai nạn nặng và tử vong (phần màu đỏ và xanh) gia tăng đáng kể so với tổng số vụ, củng cố giả thuyết rằng tốc độ cao làm tăng trực tiếp khả năng xảy ra tai nạn nghiêm trọng.

3.4. Quy trình tiền xử lý

3.4.1. Xử lý giá trị khuyết thiếu, giá trị trùng lặp và chuẩn hóa dữ liệu lỗi

Dữ liệu giao thông thực tế thường chứa nhiều giá trị không xác định, được hệ thống ghi chép ký hiệu là -1 hoặc 9 (đối với biến giới tính). Bước đầu tiên trong quy trình là chuyển

đổi các giá trị này về dạng None (Null) để các hàm thống kê của PySpark có thể nhận diện chính xác.

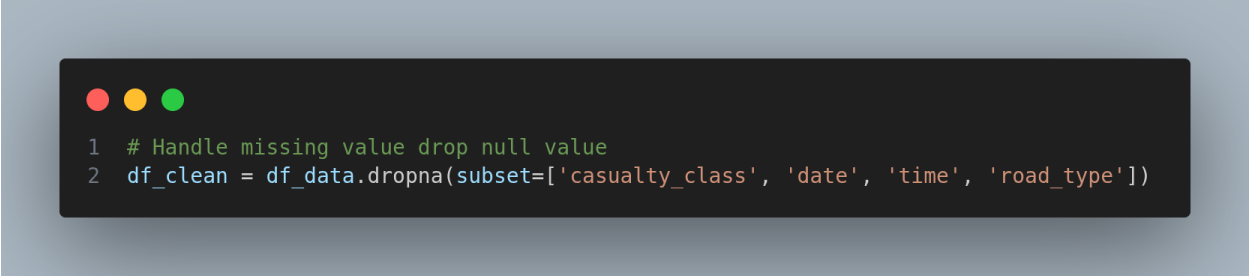
```
1 # Check missing value
2 def check_missing_val(df):
3     check_list = []
4
5     for col_name, dtype in df.dtypes:
6         condition = F.col(col_name).isNull()
7
8         if dtype in ['double', 'float']:
9             condition = condition | F.isnan(F.col(col_name))
10
11     check_list.append(F.count(F.when(condition, col_name)).alias(col_name))
12 df.select(*check_list).show(vertical=True)
```

Hình 3.10: Hàm check giá trị thiếu

Sau khi xác định được tỷ lệ khuyết thiếu, hệ thống thực hiện loại bỏ các bản ghi thiếu thông tin tại các cột trọng yếu như casualty_class, date, time và road_type do dữ liệu mất mát tương đối ít so với tổng số bản ghi chỉ chiếm một phần rất nhỏ nên có thể drop được.

```
-RECORD 0-----
collision_index      | 0
casualty_class       | 1
sex_of_casualty      | 11940
age_of_casualty      | 218553
casualty_severity    | 0
casualty_type        | 0
collision_severity    | 0
date                 | 10
day_of_week          | 0
time                  | 10
road_type             | 124
speed_limit           | 358
light_conditions     | 2621
weather_conditions   | 3350
road_surface_conditions | 17010
urban_or_rural_area   | 4837106
```

Hình 3.11: Số lượng giá trị thiếu trong từng cột



```
1 # Handle missing value drop null value
2 df_clean = df_data.dropna(subset=['casualty_class', 'date', 'time', 'road_type'])
```

Hình 3.12: Bỏ các cột có giá trị thiếu ít

Việc tồn tại các bản ghi trùng lặp có thể dẫn đến hiện tượng quá khớp và làm sai lệch các kết quả thống kê về mức độ nghiêm trọng. Hệ thống thực hiện việc loại bỏ các dòng dữ liệu dư thừa dựa trên một tập hợp các khóa chính bao gồm: mã vụ tai nạn (collision_index), phân lớp nạn nhân (casualty_class), độ tuổi (age_of_casualty) và giới tính (sex_of_casualty). Việc kết hợp nhiều trường thông tin này đảm bảo rằng mỗi thực thể nạn nhân trong một vụ tai nạn cụ thể chỉ được ghi nhận một lần duy nhất, loại bỏ các sai sót trong quá trình tổng hợp dữ liệu từ nhiều nguồn khác nhau [2]. Sau bước này, tập dữ liệu trở nên tinh gọn và phản ánh chính xác số lượng thương vong thực tế trên đường bộ.

Đối với các biến định lượng như giới hạn tốc độ (speed_limit) và tuổi nạn nhân (age_of_casualty), nghiên cứu áp dụng kỹ thuật gán giá trị bằng trung vị (Median) được tính toán thông qua hàm approxQuantile với sai số cho phép là 0.01.

Trong khi đó, các biến định danh như điều kiện ánh sáng, thời tiết và mặt đường được lấp đầy bằng giá trị yếu vị (Mode) – giá trị xuất hiện phổ biến nhất trong tập dữ liệu.

Đối với biến khu vực (urban_or_rural_area), các giá trị khuyết thiếu được gán mặc định là 3 (không xác định) để duy trì tính toàn vẹn của dữ liệu.



```
1 df_clean = df_clean.fillna({"urban_or_rural_area": 3})
```

Hình 3.13: Xử lý giá trị thiếu bằng cách điền 3 (unknown)

3.4.2. Rời rạc hóa biến thời gian

Biến thời gian gốc thường ở dạng liên tục hoặc rời rạc theo từng phút, gây khó khăn cho việc tìm ra các quy luật mang tính chu kỳ. Hệ thống thực hiện kỹ thuật rời rạc hóa (Binning) để chuyển đổi cột thời gian thành 4 nhóm đặc trưng: Buổi sáng (từ 5h đến trước 11h), Buổi trưa (từ 11h đến trước 14h), Buổi chiều (từ 14h đến trước 18h) và các khung giờ còn lại thuộc nhóm Buổi tối/Đêm. Việc phân nhóm này dựa trên đặc thù lưu lượng giao thông và điều kiện ánh sáng tự nhiên, giúp mô hình học máy dễ dàng nắm bắt mối tương quan giữa thời điểm trong ngày và mức độ nghiêm trọng của tai nạn, ví dụ như rủi ro cao hơn vào các khung giờ nhập nhem tối hoặc giờ cao điểm [1].

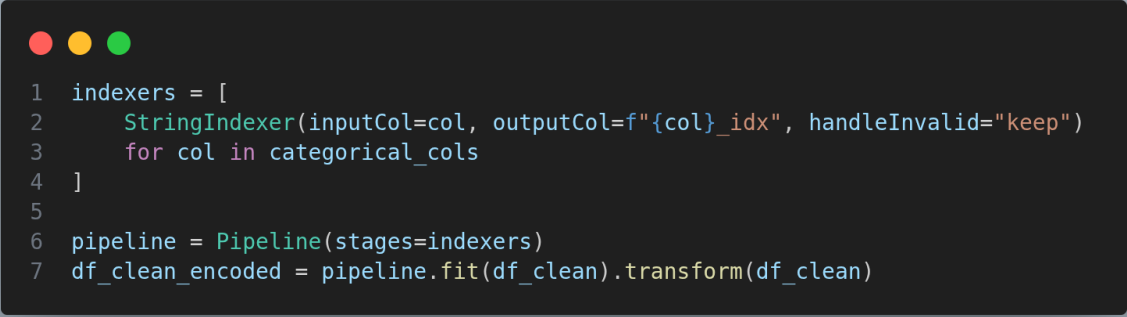


```
1 # Binning col time into 4 bins
2 # 1: morning, 2: noon, 3: afternoon, 4: night
3
4 df_clean = df_clean.withColumn("time_bin",
5     F.when((F.col("time") >=5 ) & (F.col("time") < 11), 1) # 1: Morning
6     .when((F.col("time") >=11 ) & (F.col("time") < 14), 2) # 2: Noon
7     .when((F.col("time") >=14 ) & (F.col("time") < 18), 3) # 3: Afternoon
8     .otherwise(4)
9 )
```

Hình 3.14: Phân khoảng thời gian

3.4.4. Mã hóa đặc trưng phân loại bằng *StringIndexer*

Để chuẩn bị dữ liệu cho thuật toán Random Forest, các biến phân loại dạng chuỗi hoặc số nguyên định danh cần được chuyển đổi thành các chỉ số số (Numeric Indices). Nghiên cứu xây dựng một danh sách các cột cần mã hóa bao gồm `road_type`, `weather_conditions`, `light_conditions`, `urban_or_rural_area` và biến thời gian đã phân nhóm `time_bin`. Sử dụng công cụ *StringIndexer* trong một quy trình Pipeline, mỗi giá trị trong các cột này được ánh xạ thành một con số dựa trên tần suất xuất hiện. Đặc biệt, cấu hình `handleInvalid="keep"` được thiết lập để đảm bảo rằng nếu xuất hiện các nhãn dữ liệu mới trong tương lai, hệ thống vẫn có thể xử lý mà không gây ra lỗi dừng chương trình, đảm bảo tính sẵn sàng cao cho hệ thống [4].

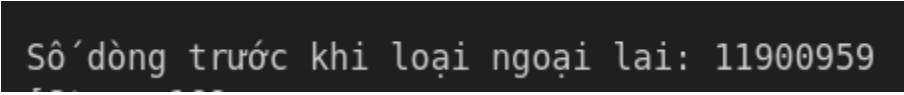


```
1 indexers = [  
2     StringIndexer(inputCol=col, outputCol=f"{col}_idx", handleInvalid="keep")  
3     for col in categorical_cols  
4 ]  
5  
6 pipeline = Pipeline(stages=indexers)  
7 df_clean_encoded = pipeline.fit(df_clean).transform(df_clean)
```

Hình 3.15: Thiết lập pipeline và *stringindexer* cho các biến phân loại

3.4.5. Xử lý *Outlier*

Số dòng trước khi xử lý ngoại lai



```
Số dòng trước khi loại ngoại lai: 11900959
```

Hình 3.16: Số dòng trước ngoại lai

Trong bước xử lý ngoại lai, dữ liệu ở hai cột `age_of_casualty` và `speed_limit` được kiểm tra bằng phương pháp IQR bằng cách tính Q1, Q3, IQR và xác định ngưỡng dưới, ngưỡng trên để nhận diện các giá trị bất thường. Sau đó, dữ liệu được lọc theo hai bước: (1) loại bỏ các

bản ghi có chứa outlier theo IQR và (2) loại bỏ các giá trị không hợp lý về mặt thực tế (ví dụ tuổi < 0 hoặc > 120 , tốc độ > 200). Cuối cùng, chương trình thống kê số lượng bản ghi trước và sau mỗi bước xử lý, đồng thời hiển thị một phần dữ liệu đã được làm sạch để phục vụ cho các bước phân tích hoặc huấn luyện mô hình tiếp theo.

Số dòng sau khi đã xử lý ngoại lai :

```
Số dòng ban đầu: 11900959
Sau khi loại outlier (IQR): 11721072
Sau khi loại giá trị vô lý: 11721072
Số dòng bị loại tổng: 179887

Xem thử dữ liệu cuối cùng:
[Stage 187:>                                     (0 + 1) / 1]
+-----+-----+-----+-----+-----+
|age_of_casualty|age_of_casualty_is_outlier|speed_limit|speed_limit_is_outlier|has_any_outlier|
+-----+-----+-----+-----+-----+
|           21|              0|         30|              0|          0|
|           48|              0|         30|              0|          0|
|           57|              0|         70|              0|          0|
|           29|              0|         30|              0|          0|
|           30|              0|         30|              0|          0|
|           30|              0|         30|              0|          0|
|           20|              0|         30|              0|          0|
|           19|              0|         40|              0|          0|
|           20|              0|         30|              0|          0|
|           17|              0|         70|              0|          0|
+-----+-----+-----+-----+-----+
```

Hình 3.17: Số dòng sau khi xử lý

3.4.6. Xử lý dữ liệu mất cân bằng

- Phân bố lớp trước khi cân bằng :

```
+-----+-----+
|casualty_severity| count|
+-----+-----+
|           1| 145378|
|           2|1874568|
|           3|9701126|
+-----+-----+
```

Hình 3.18: Phân bố lớp trước khi cân bằng

Trong bước xử lý mất cân bằng dữ liệu, tập dữ liệu sau khi xử lý ngoại lai được sử dụng để kiểm tra phân bố số lượng mẫu ở từng lớp của biến mục tiêu. Sau đó, hệ thống tính số lượng mẫu của mỗi lớp và xác định lớp có số lượng lớn nhất làm mốc tham chiếu. Dựa trên đó, trọng số cho từng lớp được tính bằng cách lấy số lượng lớn nhất chia cho số lượng của lớp tương ứng, nên những lớp có ít mẫu hơn sẽ nhận trọng số lớn hơn. Cuối cùng, các trọng số này được ghép trở lại vào tập dữ liệu gốc thông qua cột nhãn, tạo thành cột `class_weight` để hỗ trợ mô hình chú ý nhiều hơn đến các lớp thiểu số trong quá trình huấn luyện, từ đó giảm ảnh hưởng của hiện tượng mất cân bằng dữ liệu.

```

Số lượng lớp lớn nhất: 9701126
Trọng số của từng lớp:
+-----+-----+-----+
|casualty_severity| count|      class_weight|
+-----+-----+-----+
|                1| 145378|66.73035810095062|
|                2|1874568|5.175126215746775|
|                3|9701126|                1.0|
+-----+-----+-----+

```

Hình 3.19: Lớp sau khi có trọng số

3.4.7. Chuẩn hóa dữ liệu

Trong bước chuẩn hóa dữ liệu, tập dữ liệu đã được gán trọng số lớp tiếp tục được xử lý để tạo đầu vào phù hợp cho mô hình học máy. Trước hết, các biến số liên tục được gom lại thành một vector thông qua `VectorAssembler`, tạo ra cột `numeric_features_before_scaling`. Sau đó, `StandardScaler` được sử dụng để chuẩn hóa các biến số này bằng cách đưa dữ liệu về dạng có trung bình bằng 0 và độ lệch chuẩn bằng 1, giúp các đặc trưng có cùng thang đo và hạn chế việc một biến có giá trị lớn lấn át các biến khác. Sau khi chuẩn hóa xong, các đặc trưng số đã scale được kết hợp với các đặc trưng phân loại đã mã hóa trước đó để tạo thành vector đặc trưng cuối cùng trong cột `features`. Kết quả này tạo ra bộ dữ liệu hoàn chỉnh, trong đó vừa có nhãn, trọng số lớp,

các đặc trưng số trước và sau chuẩn hóa, vừa có vector đầu vào cuối cùng để phục vụ quá trình huấn luyện mô hình.

casualty_severity	class_weight	numeric_features_before_scaling	numeric_features_scaled	features
	1 66.73035810095062	[16.0,60.0]	[-0.9338632739468...	(12,[0,1,4,7,10,1...
	1 66.73035810095062	[13.0,60.0]	[-1.1053952185828...	(12,[0,1,9,10,11]...
	1 66.73035810095062	[67.0,60.0]	[1.98217978486429...	(12,[0,1,5,11],[1...
	1 66.73035810095062	[17.0,60.0]	[-0.8766859590682...	(12,[0,1,3,9,11],...
	1 66.73035810095062	[69.0,40.0]	[2.09653441462159...	[2.09653441462159...

Hình 3.20: Kết quả vector cuối cùng

3.4.8. Chia tập train/ test bằng Stratified Sampling

```
ratio = {1.0: 0.8, 2.0: 0.8, 3.0: 0.8}
df_train = df_final.sampleBy(label_col, fractions=ratio, seed=42)
df_test = df_final.subtract(df_train)
df_train.groupBy(label_col).count().orderBy(label_col).show()
```

casualty_severity	count
1	145378
2	1874568
3	9701126

Hình 3.21: Chia tập trên test bằng stratified sampling

Để chuẩn bị dữ liệu cho quá trình huấn luyện, nhóm đã tiến hành chia tập dữ liệu thành hai phần train và test. Do nhận thấy cột nhãn mục tiêu có sự mất cân bằng cực kỳ lớn, cụ thể là các ca tai nạn nhẹ chiếm tới hơn 82% trong khi số ca tử vong chỉ vỏn vẹn 1.3%, nhóm quyết định không cắt mẫu ngẫu nhiên mà áp dụng kỹ thuật lấy mẫu phân tầng (Stratified Sampling). Phương pháp này giúp ép tỷ lệ các mức độ tai nạn trong cả tập train và test luôn đồng nhất với phân bố của dữ liệu gốc, đảm bảo tập test không bị khuyết thiếu các ca tử vong hiếm gặp và giúp mô hình được đánh giá một cách khách quan nhất

Số lượng bản ghi sau khi lấy 80% dữ liệu gốc, từ 12.035.106 dòng xuống 9.375.410 dòng. Lấy chính 9 triệu dòng này làm tập train, tập test được sinh ra từ việc lấy tập dữ liệu trừ đi tập train

3.4.9. Kiểm định Chi-bình phương

Để đánh giá mức độ ảnh hưởng của các yếu tố môi trường và hoàn cảnh đến biến mục tiêu là mức độ nghiêm trọng của tai nạn, nhóm đã tiến hành áp dụng kiểm định thống kê Chi-bình phương cho các đặc trưng phân loại.

```
1 chi_assembler = VectorAssembler(  
2     inputCols=encoded_feature_cols,  
3     outputCol="cat_features_vector",  
4     handleInvalid="keep")  
5  
6 df_for_chi = chi_assembler.transform(df_train).select("cat_features_vector", label_col)  
7  
8 chi_result = ChiSquareTest.test(df_for_chi, "cat_features_vector", label_col).head()  
9  
10 p_values = chi_result.pValues.toArray()  
11 statistics = chi_result.statistics.toArray()  
12  
13 chi_df = pd.DataFrame({  
14     "Đặc trưng (Feature)": encoded_feature_cols,  
15     "Điểm Chi-Square (Statistic)": statistics,  
16     "P-Value": p_values  
17 })
```

Đặc trưng (Feature)	Điểm Chi-Square (Statistic)	P-Value
casualty_type_idx	366259.686409	0.0
urban_or_rural_area_idx	134453.606764	0.0
casualty_class_idx	111731.079887	0.0
light_conditions_idx	70692.374865	0.0
sex_of_casualty_idx	64806.565115	0.0
time_bin_idx	38128.164258	0.0
road_type_idx	34811.367126	0.0
weather_conditions_idx	8462.193331	0.0
day_of_week	7539.196065	0.0
road_surface_conditions_idx	1284.339413	0.0

Hình 3.22: Kiểm định chi bình phương

Kết quả chạy thuật toán cho thấy tất cả giá trị p-value đều bằng 0. Do dữ liệu tai nạn giao thông của nhóm có tới hàng triệu dòng, công thức Chi-bình phương sẽ liên tục cộng dồn các sai số lại với nhau. Việc cộng dồn trên lượng dữ liệu lớn như vậy đã đẩy điểm số Chi-Square (Statistic) lên cực kỳ cao, lên tới hàng chục, hàng trăm ngàn điểm. Vì điểm statistic của mọi biến phân loại đều cao, giữ lại tất cả biến phân loại.

3.5. Mô hình học máy Random Forest

3.5.1. Chọn mô hình và dữ liệu đầu vào.

Sau khi hoàn tất quy trình tiền xử lý và trích chọn đặc trưng, tiếp theo sẽ bước vào giai đoạn xây dựng mô hình học máy để dự báo mức độ nghiêm trọng của tai nạn. Do tính chất phi tuyến tính và sự phức tạp giữa các yếu tố môi trường trong dữ liệu giao thông, thuật toán Random Forest Classifier được ưu tiên sử dụng nhờ khả năng xử lý tốt các biến số đa dạng mà không bị ảnh hưởng quá nhiều bởi nhiễu. Tập dữ liệu đầu vào cho mô hình bao gồm cột đặc trưng features đã được mã hóa thành các vector toán học, cột nhãn mục tiêu casualty_severity và đặc biệt là cột class_weight. Việc tích hợp trọng số lớp là một kỹ thuật mang tính quyết định nhằm xử lý tình trạng mất cân bằng dữ liệu nghiêm trọng, giúp quá trình tối ưu hóa không bị thiên lệch về phía các vụ va chạm nhẹ chiếm đa số, từ đó nâng cao độ nhạy của hệ thống trong việc nhận diện các vụ tai nạn tử vong hoặc thương tích nặng.

3.5.2. Quy trình huấn luyện và cấu hình tham số.

Quy trình huấn luyện thông qua việc khởi tạo đối tượng RandomForestClassifier từ thư viện PySpark MLlib với tham số đã được tinh chỉnh nhằm đạt được sự cân bằng tối ưu giữa độ chính xác và khả năng tổng quát hóa. Dựa trên thực nghiệm, mô hình được thiết lập với 50 cây quyết định (numTrees=50), độ sâu tối đa của mỗi cây được giới hạn ở mức 12 (maxDepth=12). Việc cấu hình các tham số này không phải là ngẫu nhiên mà dựa trên sự tính toán kỹ lưỡng về tài nguyên và hiệu năng. Cụ thể, số lượng 50 cây là mức lý tưởng để thuật toán hội tụ và phát huy sức mạnh của phương pháp học kết hợp

(ensemble) mà không làm tăng đột biến thời gian tính toán. Trong khi đó, giới hạn độ sâu ở mức 12 đóng vai trò như một kỹ thuật cắt tỉa thiết yếu, giúp kiểm soát sự phức tạp của từng cây, ngăn chặn hiện tượng overfitting trên tập dữ liệu, đồng thời giảm thiểu rủi ro tràn bộ nhớ trong môi trường xử lý phân tán của Spark. Đồng thời, hạt giống ngẫu nhiên được cố định (seed=42) nhằm đảm bảo tính tái lập của mọi kết quả chạy. Hệ thống sử dụng trực tiếp cột class_weight làm tham số weightCol để điều chỉnh tầm quan trọng của các nhãn trong quá trình tính toán. Sau khi hoàn tất cấu hình, hàm fit được triệu gọi để mô hình bắt đầu học các quy luật tiềm ẩn từ tập dữ liệu.

```
rf = RandomForestClassifier(  
    featuresCol="features",  
    labelCol="casualty_severity",  
    weightCol="smoothed_weight", # D  
    numTrees=50,  
    maxDepth=12, # t  
    maxBins=64,  
    seed=42  
)
```

Hình 3.23: Thiết lập mô hình Random Forest với các tham số

3.5.3. Đánh giá hiệu suất mô hình.

Để đánh giá khách quan khả năng dự báo của mô hình, sử dụng công cụ MulticlassClassificationEvaluator trên một tập dữ liệu kiểm thử độc lập. Kết quả thực tế cho thấy mô hình đạt độ chính xác tổng thể ở mức cao là 81,39% và chỉ số F1-Score đạt 77,29%. Việc sử dụng cả Accuracy và F1-Score thay vì chỉ số AUC-ROC là do bản chất bất đối xứng của bài toán.

Accuracy giúp chúng ta hiểu rõ hơn về tỷ lệ đoán đúng, chủ yếu nhờ dự đoán chính xác lớp tai nạn nhẹ chiếm đa số. Tuy nhiên, nó có thể gây ngộ nhận về năng lực thực sự của hệ thống. Để khắc phục điểm này, F1-Score - trung bình điều hòa giữa độ chính xác và độ triệu hồi - được sử dụng làm thước đo kiểm chứng. Đạt được mức F1-Score 77,29% trên tập dữ liệu đa lớp phức tạp là minh chứng rõ ràng cho việc hệ thống không bị overfitting mà thực sự nắm bắt được quy luật của các nhóm thiểu số.

Phân tích sâu Ma trận nhầm lẫn cung cấp những góc nhìn đắt giá về chiến lược học của mô hình. Nhìn vào ma trận, có thể thấy hệ thống đã nhận diện thành công một số lượng lớn các ca khó: 1838785 vụ tai nạn tử vong và 60580 vụ nghiêm trọng. Đổi lại, hệ thống có xu hướng phân loại nhầm một phần các vụ tai nạn nhẹ thành tai nạn nghiêm trọng. Trong lĩnh vực an toàn giao thông, đây không phải là một khiếm khuyết mà là kết quả tích cực của kỹ thuật gán trọng số lớp. Việc đánh trọng số cao cho các nhãn thiểu số đã chủ động ép mô hình hình thành tư duy “thà cảnh báo nhầm còn hơn bỏ sót”.

Về mặt rủi ro, việc báo động giả một tình huống nguy hiểm luôn được ưu tiên chấp nhận và ít để lại hậu quả hơn việc bỏ qua một vụ va chạm có nguy cơ tử vong cao. Tính năng thiên vị có chủ đích này biến Random Forest từ một thuật toán phân loại đơn thuần trở thành một hệ thống cảnh báo sớm an toàn và có tính ứng dụng thực tiễn cao.

=====			
HIỆU SUẤT MÔ HÌNH RANDOM FOREST (FULL DATA):			
- Độ chính xác tổng thể (Accuracy): 81.08%			
- Điểm F1-Score: 77.76%			
=====			

+-----+-----+-----+			
casualty_severity_prediction	1.0	2.0	3.0
+-----+-----+-----+			
	1 1999	7258	19624
	2 4032	60580	310813
	3 4542	97491	1838785
+-----+-----+-----+			

Hình 3.24: Kết quả huấn luyện mô hình

CHƯƠNG 4: ĐÁNH GIÁ HỆ THỐNG VÀ THẢO LUẬN KẾT QUẢ

4.1. Đánh giá kiến trúc tổng thể của hệ thống

Quá trình tiền xử lý bao gồm tích hợp và xử lý bộ dữ liệu từ năm 1979 đến năm 2025 một cách ổn định. Việc sử dụng cơ chế spark để có thể chia nhỏ dữ liệu ra và xử lý nhanh hơn

và việc lưu trữ dưới dạng file parquet giúp tối ưu hóa dung lượng đĩa và đẩy nhanh tốc độ đọc, ghi dữ liệu từ đó các thao tác tiền xử lý khác được đẩy nhanh hơn. Việc tiền xử lý đã loại bỏ đi các thông tin dư thừa, nhiễu và chuẩn hóa lại để phục vụ cho mô hình học máy

4.2. Khả năng dự báo và ý nghĩa thực tiễn

Kết quả thực nghiệm cho thấy mô hình Random Forest sau khi được tối ưu hóa đã đạt Độ chính xác tổng thể (Accuracy) ở mức rất ấn tượng là 81.39% và chỉ số F1-Score đạt 77.29%. Trong bối cảnh bài toán dự báo tai nạn giao thông có cấu trúc dữ liệu cực kỳ mất cân bằng và mang tính phi tuyến tính cao, việc duy trì được chỉ số F1-Score ở mức xấp xỉ 80% là một tín hiệu đặc biệt tích cực. Điều này chứng tỏ kỹ thuật gán trọng số lớp (class_weight) đã phát huy tác dụng triệt để, giúp mô hình không bị rơi vào bẫy "học vẹt" (chỉ dự đoán lớp tai nạn nhẹ chiếm đa số) mà thực sự phân bổ năng lực tính toán để nhận diện các lớp thiểu số nghiêm trọng. Đáng chú ý, việc hệ thống có xu hướng phân loại nhằm một lượng nhất định các vụ tai nạn nhẹ thành tai nạn nghiêm trọng không phải là một khiếm khuyết, mà là một chiến lược ưu tiên an toàn theo nguyên tắc "thà cảnh báo nhầm còn hơn bỏ sót". Khả năng nhận diện nhạy bén này được minh chứng rõ qua các thử nghiệm dự báo, khi hệ thống có thể phát hiện và đưa ra cảnh báo nguy cơ tử vong (Nhãn 1) với độ tự tin áp đảo so với các nhãn khác dựa trên bối cảnh môi trường. Về mặt ý nghĩa thực tiễn, mức hiệu năng xuất sắc này khẳng định mô hình hoàn toàn đủ độ tin cậy để các cơ quan quản lý sử dụng như một lời tính toán rủi ro độc lập. Hệ thống có thể được tích hợp để tự động đưa ra các quyết định phân luồng, kích hoạt biển báo điện tử hoặc yêu cầu giảm tốc độ chủ động trên các đoạn đường có điều kiện thời tiết khắc nghiệt, từ đó góp phần trực tiếp vào việc giảm thiểu thiệt hại về người.

4.3. Hạn chế và hướng phát triển tương lai

Dù đã đạt được năng lực nhận diện rủi ro với độ chính xác lên tới 81.39%, hệ thống vẫn tồn tại một số hạn chế nhất định phản ánh bản chất vô cùng phức tạp và mang tính ngẫu nhiên của các vụ va chạm đường bộ thực tế. Khoảng trống gần 20% sai số còn lại chủ yếu đến từ việc các đặc trưng đầu vào hiện tại (thời tiết, ánh sáng, tốc độ giới hạn, loại đường) mới chỉ mô phỏng được các nguyên nhân khách quan. Trong khi đó, hệ thống

chưa thể bao quát hết các yếu tố cốt lõi mang tính chủ quan từ phía hành vi con người như độ tỉnh táo, nồng độ cồn trong máu hay thời gian phản xạ của tài xế . Bên cạnh đó, về mặt hạ tầng kỹ thuật, việc phân tán các tác vụ huấn luyện với tập dữ liệu hàng chục triệu bản ghi vẫn tạo ra những nút thắt (bottleneck) về bộ nhớ, biểu hiện qua các cảnh báo vượt quá kích thước tác vụ (task size) trong môi trường PySpark khi lưu trữ mô hình xuống đĩa. Trong tương lai, nghiên cứu hướng tới việc thu thập và làm giàu thêm các chiều dữ liệu về hành vi lái xe và tình trạng lưu lượng phương tiện thời gian thực. Đồng thời, kiến trúc mô hình có thể được nâng cấp thông qua việc thử nghiệm các thuật toán tăng cường độ dốc (Gradient Boosting) mạnh mẽ hơn như XGBoost hoặc LightGBM, kết hợp cùng nền tảng xử lý luồng Spark Streaming để tiến tới xây dựng một hệ thống cảnh báo giao thông trực tuyến với độ trễ thấp và độ chính xác toàn diện hơn.

TÀI LIỆU THAM KHẢO

- [1] S. Garcia, J. Luengo, and F. Herrera, “Data Preprocessing in Data Mining.”
- [2] J. Han, M. Kamber, and J. Pei, “Data Mining: Concepts and Techniques”.
- [3] H. He and Y. Ma, “Imbalanced Learning: Foundations, Algorithms, and Applications”.
- [4] Apache Spark Team (2026), “MLlib: Main Guide - Extracting, transforming and selecting features”.
- [5] L. Breiman, *Random Forests. Machine Learning*, 45(1), 5-32. 2001.